

A Project Proposal Report on

Hangman

A Project Report Submitted for the Course

Object-Oriented Programming (OOP)

Bachelor in Computer Science

Submitted by:

Abhilasha Shrestha, WU0257723

Date:

08/10/2026



King's College Nepal

Bijulibajar, Kathmandu

ABSTRACT

”Hangman” is a console-based word guessing game developed in C++ that demonstrates the practical application of Object-Oriented Programming (OOP) principles. The primary objective of the project is to design an interactive game in which the player attempts to guess a hidden word by suggesting letters within a limited number of attempts. The game makes use of an abstract base class to define a general interface for gameplay, which is then extended through inheritance by a derived class implementing the complete game logic. The internal state of the game, such as the target word, the current progress, and the remaining attempts, is kept encapsulated and accessible only through dedicated member functions. Polymorphism is demonstrated through the overriding of a virtual function responsible for rendering an ASCII-art hangman figure that updates progressively with each incorrect guess. The technologies used for this project include the C++ programming language along with a standard compiler and development environment. With a simple, well-structured design, the project aims to reinforce core object-oriented concepts while producing an engaging and replayable console game. This project therefore illustrates how abstraction, encapsulation, inheritance, and polymorphism can be combined to build a small but complete interactive application.

Keywords: *Hangman, Object-Oriented Programming, Abstraction, Encapsulation, Inheritance, Polymorphism*

TABLE OF CONTENTS

Abstract	i
Table of Contents	ii
List of Figures	iii
List of Tables	iv
1 INTRODUCTION	1
1.1 PROBLEM STATEMENT	1
1.2 PROJECT OBJECTIVES	1
1.3 SCOPE AND LIMITATIONS	2
2 LITERATURE REVIEW	3
3 METHODOLOGY	4
3.1 SOFTWARE DEVELOPMENT LIFECYCLE	4
3.1.1 Requirement Analysis	4
3.1.2 Design	4
3.1.3 Coding	4
3.1.4 Testing and Evaluation	5
3.2 TOOLS TO BE USED	5
3.3 TECHNOLOGIES TO BE USED	5
4 SYSTEM DESIGN	7
4.1 USE CASE DIAGRAM	7
4.2 SEQUENCE DIAGRAM	8
4.3 CLASS DIAGRAM	9
5 PROPOSED DELIVERABLES	10
6 PROJECT TASK AND TIME SCHEDULE	11

LIST OF FIGURES

1	Use Case Diagram	7
2	Sequence Diagram	8
3	Class Diagram	9

LIST OF TABLES

1	Time Schedule	11
---	-------------------------	----

1. INTRODUCTION

This project carries the motive of providing a simple, engaging, and educational implementation of the classic Hangman word guessing game, developed using the C++ programming language with an object-oriented design. In this section, problems, motivations, objectives, project scope and limitations will be described in detail.

1.1 PROBLEM STATEMENT

Many beginner and intermediate level programming exercises focus purely on procedural logic, without properly applying object-oriented design principles such as abstraction, encapsulation, inheritance, and polymorphism. As a result, learners often struggle to translate theoretical OOP concepts into practical, working applications.

Additionally, most simple console-based games available as learning resources lack visual feedback, making them less engaging for the player. A game without a proper structured design also becomes difficult to extend or maintain when new features are to be added.

This creates a need for a small but well-structured project that not only functions as a playable game but also clearly demonstrates sound object-oriented programming practices.

1.2 PROJECT OBJECTIVES

To address the problems stated above in problem statement section 1.1, a structured, object-oriented approach to game development is required. So, with the purpose of providing a practical demonstration of OOP concepts through an interactive project, the Hangman game is introduced with the following objectives:

1. **Design an interactive console-based game:** The primary objective of this project is to provide a functional and engaging Hangman game playable through the console.
2. **Demonstrate object-oriented programming principles:** The project applies abstraction, encapsulation, inheritance, and polymorphism in a practical setting to reinforce theoretical understanding.

3. **Provide visual feedback to the player:** An ASCII-art hangman figure is rendered and progressively updated with each incorrect guess, improving the overall gameplay experience.
4. **Build a reusable and extensible codebase:** By structuring the project around a base class and a derived class, the design allows future features to be added with minimal changes to existing code.

1.3 SCOPE AND LIMITATIONS

In this project, a console-based Hangman game will be developed in such a way that a single player can interact with the system by guessing letters of a hidden word until the word is completely revealed or the maximum number of incorrect attempts is reached.

The scopes of this project are:

1. The target audience includes beginner to intermediate level programmers and students who wish to understand practical applications of OOP in C++.
2. The game can be compiled and run on any system with a standard C++ compiler, making it platform independent at the source-code level.
3. The modular class-based design allows the project to be extended with additional features such as word banks, categories, or a graphical interface.

The limitations of this project are:

1. The current version supports only a single predefined word rather than a dynamic word bank.
2. The game is limited to a console-based text interface and does not currently include a graphical user interface (GUI).
3. The game supports only single-player mode.

2. LITERATURE REVIEW

In this section, we have reviewed some of the existing implementations and approaches related to the Hangman game that are commonly available.

Most beginner-level Hangman implementations found in online tutorials and coding exercises are written using purely procedural programming, where the word, the guessed letters, and the attempt counter are handled through global variables and standalone functions. While such implementations are functional, they do not demonstrate any object-oriented structure, making the code harder to extend or reuse in larger projects. In our proposed implementation, we address this by structuring the game using an abstract base class and a derived class, clearly separating the interface from the implementation.

Other implementations of Hangman found in academic resources sometimes apply object-oriented concepts but limit themselves to encapsulation alone, without demonstrating abstraction or polymorphism. This results in an incomplete representation of OOP principles. In our project, we implement all four pillars of OOP together, using a pure virtual function in the base class that is overridden in the derived class, thereby demonstrating abstraction and polymorphism alongside encapsulation and inheritance.

3. METHODOLOGY

We will be adopting the following methodologies for a smooth development process to meet the requirements of the project and not deviate from it.

3.1 SOFTWARE DEVELOPMENT LIFECYCLE

We will be following the incremental model of software development as a framework for this project. The incremental model of software development is an approach where the development process is broken down into smaller increments, each involving different phases such as Requirement Analysis, Design, Coding, Testing and Evaluation. In this model, each increment delivers a partial but usable version of the project.

In the first increment, we will be developing the core product having major functionalities such as the abstract base class definition, the derived class implementation, the word guessing logic, and the basic ASCII hangman rendering.

In the subsequent increments, we will be adding more functionalities such as a dynamic word bank and difficulty levels to satisfy the requirement specification.

3.1.1 Requirement Analysis

This is the first step of an incremental model where requirements will be collected and analysed. The outcome of this phase will be a System Requirement Specification (SRS) document.

3.1.2 Design

In this phase, we will be designing the system based on the System Requirement Specification (SRS). We will be developing a use case diagram, a sequence diagram, and a class diagram to represent the structure and behaviour of the system.

3.1.3 Coding

The functionalities as specified in the System Requirement Specification (SRS) and as modelled during the design phase, will be developed through code in this phase. The

outcome of this phase will be a working product with the functionalities as specified in the document.

3.1.4 Testing and Evaluation

In this phase, different components of the system, such as the guessing logic and the hangman rendering function, will be tested individually. The entire system will also be tested to assure that the functionalities provided by the system work smoothly as specified.

3.2 TOOLS TO BE USED

We will be using the following tools throughout this project to streamline the development process.

TRELLO:

Trello is a web-based project management tool that we will be using to organize and manage individual tasks.

GIT AND GITHUB:

Git is a distributed version control system that will be used in this project to manage different versions of the project. GitHub is a platform that uses git for version control. We will be using GitHub to host the project repository and track changes.

VS CODE:

VS Code is a code editor that will be used as a primary code editor for development tasks.

GCC/G++ COMPILER:

GCC/G++ is a widely used compiler for compiling and running C++ programs. We will be using it for building and testing our application.

3.3 TECHNOLOGIES TO BE USED

We will be using the following technology during the project development:

C++:

C++ is a general-purpose, object-oriented programming language that supports classes,

inheritance, and polymorphism. We will be using C++ as the sole programming language for implementing the Hangman game, making use of its object-oriented features to structure the project.

4. SYSTEM DESIGN

This section provides a comprehensive overview of the proposed system's architecture focusing on three key diagrams: the Use Case diagram, the Sequence diagram, and the Class diagram. These diagrams illustrate the core interactions, behaviour, and structure of the system.

4.1 USE CASE DIAGRAM

A use case diagram is a visual representation of the interactions between an actor (in our case, the Player) and the system under consideration. It shows the different use cases or functionalities provided by the system, such as starting the game, guessing a letter, viewing the hangman figure, and viewing the game result.

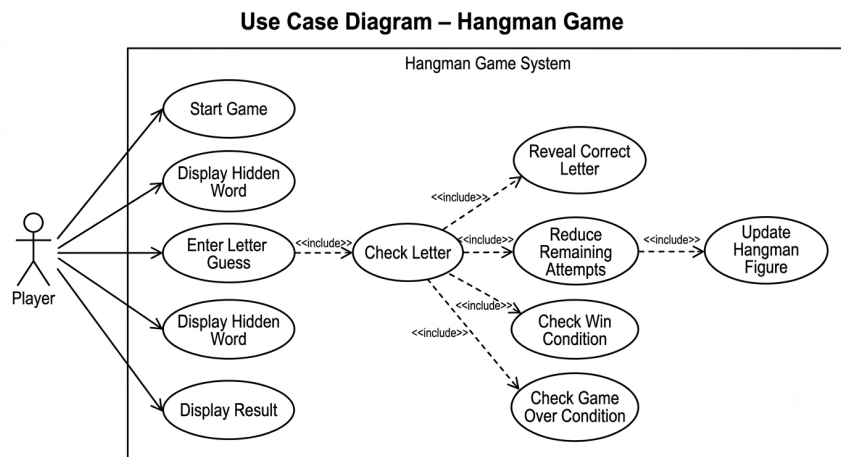


Figure 1: Use Case Diagram

4.2 SEQUENCE DIAGRAM

A sequence diagram is a visual representation that shows how objects in the system interact with one another over time, in the sequence in which those interactions occur. It illustrates the flow of messages between the Player, the HangmanGame object, and its member functions, from the start of the game through to the win or loss condition.

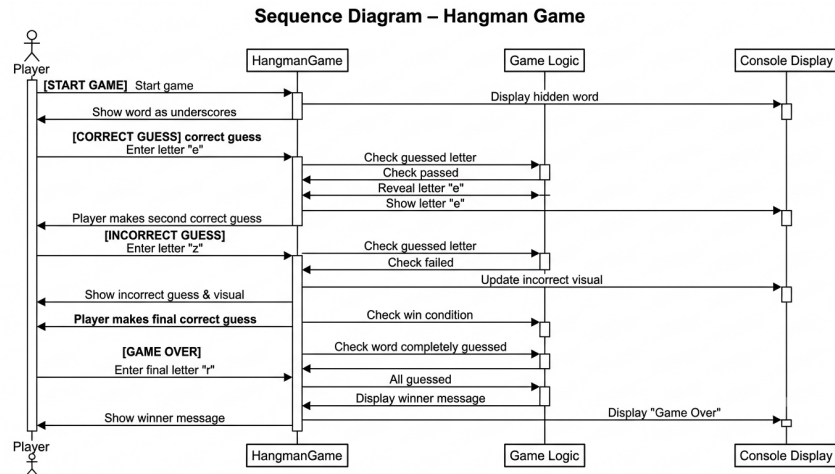


Figure 2: Sequence Diagram

4.3 CLASS DIAGRAM

A class diagram is a visual representation of the classes within a system and the relationships between them, such as inheritance. It is commonly used in object-oriented design to model the structure of a system. The major classes involved in our system are the abstract Game class and the derived HangmanGame class. The relationship between these classes is shown in the given class diagram.

Class Diagram – Hangman Game

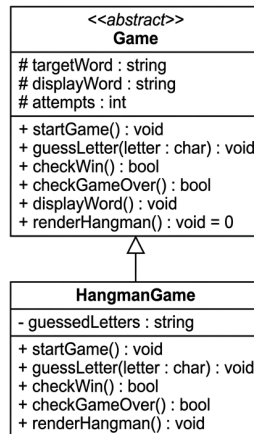


Figure 3: Class Diagram

5. PROPOSED DELIVERABLES

The project will deliver a fully functional, console-based Hangman game built using object-oriented C++. The player will be able to run and play the application directly through the command line on any system with a compatible C++ compiler. There is essentially one primary user of this application: the Player, who guesses letters of the hidden word within a limited number of attempts.

In the application, the player is able to guess letters one at a time. Correct guesses reveal the corresponding positions in the hidden word, while incorrect guesses reduce the remaining attempts and progress the ASCII hangman figure.

The system will track the number of remaining attempts and will declare the player a winner if the full word is guessed, or declare the game over if the attempts run out.

6. PROJECT TASK AND TIME SCHEDULE

The project time schedule is designed, considering the requirements and constraints involved. The project schedule is designed such that the Design phase is given adequate importance so that the object-oriented structure of the project is properly planned before coding begins.

Table 1: Time Schedule

Phase	Description	Duration
Requirement Analysis	Collecting and analysing requirements, preparing the SRS document	1 week
Design	Preparing use case, sequence, and class diagrams	1 week
Coding	Implementing the abstract base class, derived class, and game logic	2 weeks
Testing and Evaluation	Testing individual components and the complete system	1 week